CIS 4910

Computer Science & Engineering Project — Capstone

WEEK 1 • FALL 2026

Professor Scot Hollingsworth

scoth@usf.edu • Office Hours by appointment (Microsoft Teams)

Bellini College of Artificial Intelligence, Cybersecurity and Computing

WELCOME TO FALL 2026

This is the same hard, rewarding course, with major updates this semester for the engineers you're about to become.

I've taught this capstone for years. The team structure, the engineering process, and the standard for what counts as a real, working project **HAS NOT** changed. What **HAS** changed is the toolset every one of you will use in this class and on the job the week after graduation and that changes how I want you to work this semester.

Teams & ownership

3–4 person teams, real sponsors, real deliverables. This part never changes.

AI is assumed

You are expected and encouraged to use AI tools well and to disclose exactly how they are used.

Higher bar, same process

Requirements → design → build → test. AI changes the speed, not the rigor.

FIRST DAY

Agenda for Today

1. Welcome and a quick framing video
2. Who I am, and how this class is structured
3. Goal for this class
4. AI Impact to your future
5. AI Usage policy for this class
6. Who is an Engineer
7. Requirements, Specifications & Time tracking
8. Status reports
9. Syllabus walk through
10. Project selection
11. Next Steps

A Software Failure: Ariane 5

1. A flight computer converted a 64-bit number to 16 bits without checking it would fit — a runtime error.
2. The error halted the primary flight software.
3. The backup computer hit the identical bug and halted too.
4. Both computers sent diagnostic data to the main flight computer.
5. The main computer misread that data as real flight data and fully deflected the nozzle.
6. The rocket veered violently, broke apart, and was destroyed — about 37 seconds after launch.

LINK: https://www.youtube.com/watch?v=5tJPXYA0Nec



One unchecked data conversion. A $370M rocket. No human caught it before launch.

This is still the lesson: requirements, specs, and test plans exist because people miss things. AI-generated code can fail the same way — silently, at the worst moment.

YOUR INSTRUCTOR

Professor Scot Hollingsworth, I do not hold a PHD!

- Adjunct Professor at USF for over a decade
- BS Electrical Engineering, Oklahoma State University (1984)
- MS Computer Science — Information Security, Nova Southeastern (2007)
- Principal Consultant, Enterprise & Data Architecture; IT Analytics & Security, Inc.
- Career path: DoD intelligence → GPS-tracking founder → Director of IT, Verizon → independent consulting, IT Architect
- FDLE-Certified Reserve Deputy, Hillsborough Co. Sheriff's Office; certified instructor in Driving, Defensive Tactics & PIT
- Airline Transport Pilot
- Cave Diver



"40+ years across engineering, intelligence, and consulting, most of it spent fixing what 'good enough' builds left behind."

Don't believe everything Google says about me.

SOME PERSPECTIVE

My Senior Project, 1984

OSU_Class_project_1984.pdf

- Built an Optical Time Domain Reflectometer at Oklahoma State, EE program
- Hewlett-Packard purchased the design later that year as a test stand for one of their products
- I graduated into a starting salary of $24,500/year (roughly $74,000 in today's dollars) at E-Systems, doing government intelligence work
- I hope your senior project opens a door the way mine did. The AI tools are new, but the payoff of building something real hasn't changed

$24,500

starting salary, 1984

≈ $74,000

in today's dollars

Basic Rules of Behavior

Professional behavior and language (At All Times)

Professional language, as if you were on a company team because in many cases, you are. Zero profanity.

Respect always, disagree with the work, not the person

Push back on the process, the product or the method. Never make it personal.

Peer evaluation is real

Your team will be evaluated by other teams, and you will evaluate other projects for a grade.

Attendance is not “Required” but “strongly suggested”

Yes, I take role every week for a host of reasons. Your final is based on information in class.

SET THE TONE

Your Goal for This Class

Work on a project you actually find fun or cool.

- Meet new people you'll actually want as professional references
- Learn something genuinely new not just a tutorial, something you had to fight for
- Build a project you can speak about fluently in an interview
- Build real self-confidence, the kind that comes from shipping something that works

COURSE PURPOSE

What This Course Is About

Engineer Like an Engineer

- Product development & engineering design
- Critical thinking over copy-paste solutions

Learn Something On Your Own

- A required new skill, not a given one
- That struggle is the point and it feels great once you're through it

A Simulation of the Real World

- Real sponsors, real expectations, real (if smaller) pressure
- Teamwork in a space where no one gets fired

THE LANDSCAPE YOU'RE GRADUATING INTO

What AI Has Actually Changed for a New Computer Engineer

NO HYPE, NO PANIC

The Honest Picture, Mid-2026

Two things are true at the same time, and you need to plan around both of them.

⚠ Entry-level hiring has tightened

- Hiring for developers aged 22–25 has dropped sharply since 2022 as agentic tools absorb routine, boilerplate-level coding work
- Job postings for pure “translate spec into code” roles are shrinking
- The bar for what counts as junior-level competence has moved up

📈 Demand for AI-fluent engineers is rising

- Job postings requiring AI coding tool experience grew roughly 340% in one year (2025)
- BLS (Bureau of Labor & Statistics) still projects double-digit growth in software roles through the early 2030s, but not in a legacy environment
- Postings increasingly reward people who can architect, evaluate, and orchestrate not just type

WHAT'S ACTUALLY CHANGING

From Writing Code to Directing Systems

Old model

Engineer writes most lines of code by hand, debugs syntax errors, hand-builds test scaffolding to a QA team.

New model

Engineer defines the problem precisely based on Customer input, directs an AI agent through implementation, and rigorously evaluates what comes back, often building QA Bots.

Requirements / Specification

Writing requirements & Specs precise enough that an agent can't misread them.

Orchestration

Directing AI tools through multi-step work: plan, delegate, check, repeat.

Evaluation (CODE REVIEW !)

Reading generated code critically. Catching what's subtly wrong before it ships.

Judgment

Knowing what NOT to hand to a model : security, ethics, ambiguous tradeoffs.

WHAT ACTUALLY WORKS

What the Best Engineers Do Differently

- They treat AI tools as a force multiplier on their own skill, not a replacement for understanding the system
- They still read every line an agent produces before it ships, the same way you'd review a teammate's pull request
- They use AI heaviest on the parts of the job that were always grunt work, boilerplate, test scaffolding, first-draft documentation and spend the time it frees up on architecture and tradeoffs
- Large-scale developer surveys consistently find heavy AI users reporting more time in productive flow, not less when the tool is used deliberately, not as a crutch



The skill that's becoming MORE valuable, not less:

Knowing your domain well enough to tell when the AI is confidently wrong.

What Makes a “Good” Project?

Four things that make a project “Good” or “Exceptional”

Problem Solution

The project must solve a business problem as defined by the sponsor

Operational

The project must work without errors and be demonstratable to the class and sponsors

Documented

Must have proper documentation as defined by industry and in the assignment rubric

Team Knowledge

Each member of the team must be able to explain all aspects of the project in detail

READ THIS SECTION CAREFULLY

AI Usage Policy for This Course

Use it. Use it well. Disclose it every time.

PER THE SYLLABUS & USF REGULATION 3.027

The Basic Policy



AI tools ARE allowed & Encouraged; Not Required

- Claude, ChatGPT, Copilot, Cursor, and similar tools, use whatever helps you build a better project
- Open-source materials and libraries are encouraged, just follow their licenses
- There is no penalty for using AI well
- Must always disclose AI Usage
- USF Does not pay for “Token” / AI Access



Disclosure is NOT optional

- Any code, text, or design generated by an LLM or other tool (AI / Other) must be noted as such, **every time & ALWAYS**
- Presenting AI-generated work as entirely your own - even by omission - is an academic integrity violation
- This applies to individual work AND team deliverables

MAKE THIS EASY ON YOURSELF

How to Disclose AI Use — Concrete Examples

In code

A short comment or commit note: "Function scaffolded with Claude Code, logic reviewed and modified by [name]."

In written documents (SRS, DDD, reports)

A one-line note in the document history or references: "Section 3.2 drafted with ChatGPT assistance, edited by the team." (Details included in reference section of each document) Preference is given to noting AI Usage and details as a APA reference in such documents

In your individual status reports

*State plainly which tools you used and for what: drafting, debugging, research, brainstorming, editing. For status reports the **preference** is "in Your Own Words"!*

WHERE THE REAL LINE IS

Disclosure Doesn't Excuse Not Understanding It

Noting that you used AI is necessary, but it does not satisfy the requirement that **every** team member can explain every part of the project. That standard is unchanged from past semesters.



Acceptable

- “Claude generated the first draft of our REST API; Bob rewrote the error handling and can explain every endpoint.”
- Using AI to learn an unfamiliar framework faster, then demonstrating that you understand it.



Not acceptable

- Submitting AI output, you can't explain, when asked in a team presentation.
- Letting one teammate run everything through AI without the rest of the team understanding the result.

TEAM STRUCTURE

Developing Your Team

3–5 members per team (usually 4)

You'll get a chance to pick a project you want. Don't be discouraged if it's not your first choice.

Shared leadership

Each person should lead a segment of the project. Leading is a privilege and a responsibility, not something to take lightly, and not something to dodge.

Team assignments are final

Once a team is set, it doesn't change. Real changes only happen because a student drops or joins late, handled by the instructor only.

WEEK 1 ACTION ITEM

The First Team Meeting

1. Get to know each others likes, dislikes, what each person wants out of this class
2. Talk openly about strengths, weaknesses, and specifically your fears about the project. Be honest! Write them down
3. Review deliverables and assignments together so everyone starts from the same page
4. Set a standard meeting time no less than weekly preferred bi-weekly. Don't be late, and don't make excuses for missing it
5. Face-to-face is better when possible; use Teams for quick check-ins between meetings

A QUICK QUIZ

Who Is an Engineer?

Who is a doctor or lawyer?

(a) Graduated from an accredited medical or law school

(b) Passed the board exam or bar exam

(c) Anyone who wants to add the title to their name

Correct answer: (a) and (b). If you said (c), we need to talk; you could wind up in a special set of bracelets!

Who is an engineer?

(a) Graduated from an EAC ABET-accredited program

(b) Passed the Professional Engineering (PE) exam

(c) Anyone who wants to add the title to their name

Correct answer: it depends on the state, but generally (c) the title "Engineer" isn't legally locked down in most contexts.

THE TITLE KEEPS EVOLVING

The Newest Flavors of “Engineer”

“Prompt Engineer” showed up almost overnight a few years ago. It's already evolving again, which tells you something about how fast titles in the engineering field are changing

GenAI Engineer

Builds applications and products on top of large language models.

AI Orchestration Specialist

Designs systems where multiple AI agents hand off work to each other.

MLOps Specialist

Deploys, monitors, and maintains machine learning systems in production.

Prompt Engineer

Still exists, but increasingly folded into broader AI engineering roles; rarely seen any longer.

These titles (just a few) are growing in job postings faster than traditional software roles, but the core engineering skills underneath haven't changed, in fact they have become more important than ever!

LAND THE POINT

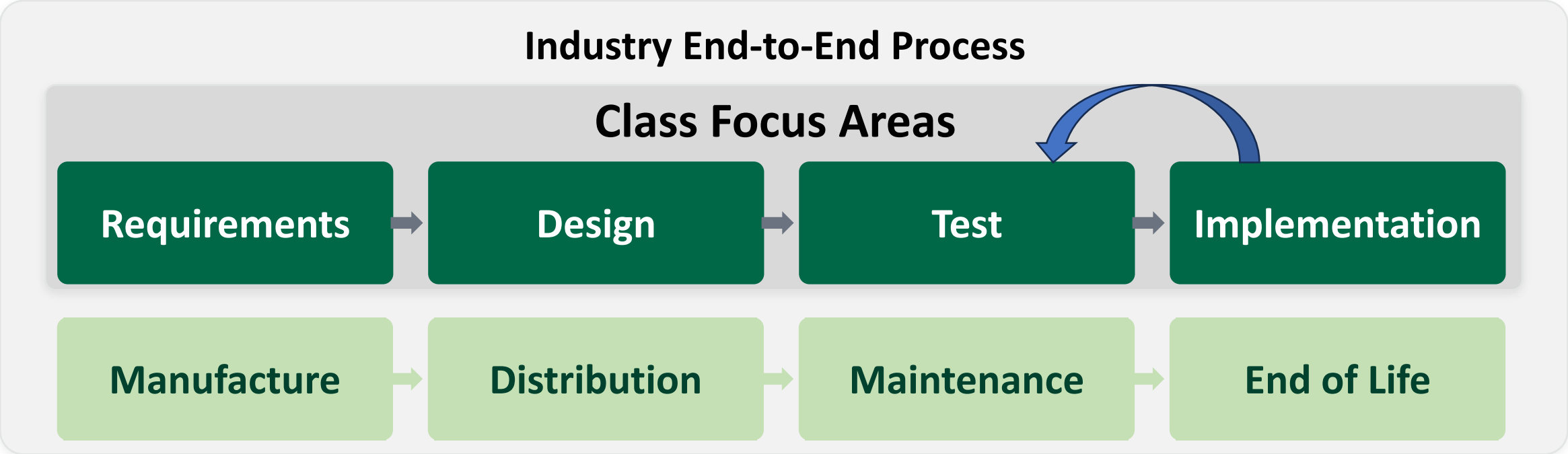
Who Is an Engineer, Really?

Someone who solves problems using engineering methods.

Requirements. Design. Implementation. Test. Whether you're hand-writing every line or directing an AI agent through the work, that process is what makes you an engineer rather than someone who just got lucky with output. That's exactly what this class will teach you.

REMEMBER THIS SLIDE

The Engineering Process



Validation & Verification run alongside every stage; are we building the right thing, and are we building it right?

This class focuses on Requirements → Design → Implementation → Test (highlighted above). AI tools can accelerate Implementation dramatically, but they don't get to skip Requirements, Design, or Test. Those stages are where engineering judgment lives, and they're graded the same way they've always been graded.

EVERYTHING STARTS HERE

Requirements, Specifications & Time

Requirements define the business problem. Get this wrong and no amount of AI-assisted speed in implementation will save the project.

STEP ONE, EVERY TIME

Requirements

The first step in any project: meet with your sponsor ASAP and define what they actually need; in their language, not yours. Requirements are usually in Business Language; Specs are in Technical speak!

Problem space, not solution space

Understand the problem extremely well before you touch a design. Requirements describe **what** the customer needs and wants, **not how** you'll build it. Develop the EPIC, then Features and finally stories.

Requirements must be:

1. Numbered
2. Measurable
3. Unique
4. In priority order

Final Exam Notes here!

Requirements vs. Assumptions

Assumptions are givens

"The software must be written in Java" is an assumption/constraint, not a requirement. Assumptions often drive specifications, the HOW of a requirement.

Requirements are things you will build, the WHAT.

Can requirements change mid-project?

Short answer: absolutely yes. We'll discuss this in depth later.

You must keep a clear history of changes to requirements and WHO changed them and why! This matters as much when an AI tool suggests the change as when a human does.

THE FORMAT WE'LL USE

What Does a User Story Look Like?

As a (role), I want (something), so that (benefit)

All Requirements & Specs in this class are to be written as "USER STORIES" (Above)

Requirement

"As a user, I want to post pictures so that I can share them with my friends."

"As an admin, I want to review posted pictures so that I can remove inappropriate ones."

Acceptance Criteria

How you know a story is "done." This is also your starting point for writing test cases.

"The ability to add new pictures to a given post and the ability to remove them."

Test Case

Given an admin is on the review dashboard,
When they click the "Remove" or "Delete" button on a specific picture,
Then the system must prompt them with a confirmation dialog.
When the admin confirms the removal,
Then the picture must be immediately hidden from the public feed and marked as deleted in the database.

THE FORMAT WE'LL USE

Requirements Can Have "Levels"

V1

L1 - High

"As a user, I want to post pictures so that I can share them with my friends."

V2-V3

L2 - Direct

" As a normal user, I want to be able to add, delete, and update pictures which are saved to a user specific location so that I can share them with my friends

L2 - Direct

" As a normal user, I want to be able to sort my pictures so I can group them into categories.

V4

L3 - Detail

" As a normal user, I want to be able to sort my pictures alphabetically so that I can group them into small categories.

" As a normal user, I want to be able to sort my pictures by date so that I can group them into small categories.

" As a normal user, I want to be able to sort my pictures people in the pictures so that I can group them into small categories.

Requirements→ Stories → Manageable Stories

Business Requirement

The company must be able to track peoples Time for each project they work on.

Engineering Requirement (13 SP's)

As an owner, I want to track each person's time so that I can track project costs of FTE's

Individual Stories (1-3 SPs)

As a dev, I want a web page that allows a username and time value to be input and saved in a DB so that their time can be tracked.

Individual Stories (1-3 SPs)

As a DB manager, I want a database that allows information about time, cost and assignments to be stored so that I can track individual time as assigned to each project.

Individual Stories (1-3 SPs)

As a CIO, I want to be able to run a report that shows me how much time a given employee spent on any specific project so that I can estimate run rate properly.

THE INVEST CHECKLIST

What Makes a Good User Story (INVEST)

I Independent

Can be developed and tested on its own and valued independently.

N Negotiable

Not a fixed contract, a placeholder to discuss, develop, and refine.

V Valuable

Provides clear value to the user, customer, or stakeholder.

E Estimable

Has clear acceptance criteria, broken down small enough to estimate effort.
Rule of 3

S Small

Fits comfortably into a single sprint.
NOTE: All sprints in this class are 1 weeklong.

T Testable

Acceptance criteria are verifiable by testers or business users.

A CRITICAL DISTINCTION

Requirements vs. Specifications

Requirement (the WHAT)
Stated by the customer

"Create a database of customer records, indexed by last name."

Tells the builder what is being requested.

Specification (the HOW)
Usually translated by the Engineer

"Create a Snowflake database in 3rd normal form, with customer records indexed by last name and zip code to support easy searches which also supports a modern Data Warehouse."

Tells the builder "how" a requirement is implemented. Does NOT demonstrate the detailed design.

A PROFESSIONAL HABIT

Time Tracking

Regardless of your employer, you will almost always be expected to track and submit weekly timesheets. This course is no different.

Required with every submission

Timesheets accompany every individual and team submission including the mid-term and final project. **USE THE TEMPLATE IN CANVAS**

Two categories

Expense (OPX) and Capital (CAPX). A template and example are posted on Canvas. I will review these items in detail in week 2 in detail.

What Is Time Tracking?

USE THE TEMPLATE IN CANVAS!

Your time really falls into only one of two categories:



Expense (Taxable) (OPEX)

- Research
- Requirements
- Specifications
- High Level Design
- Fixing Bugs
- Production Updates
- QA
- PTO



Capital (Tax deductible) (CAPX)

- Detailed Design
- Code
- Code Review
- Unit Test
- User Testing

Lying on a timesheet is cause for dismissal at almost every organization. Review the template on Canvas → Files. Providing false information on the timesheet is an academic violation issue!

Communicate with me!

How do I know how the project is moving?

Individual & Team Status Report

Status reports help team communicate, helps keep me informed about issues and helps me ensure you are moving in the proper direction.

COMMUNICATING PROGRESS

What a Status Report Is For

A status report is one-way developers communicate progress. In Agile, the other way is the daily stand-up.



Goals

What were the goals for this time period?



Accomplished

What was actually accomplished? Reflect honestly on goals missed, with a “recovery plan.”



Hours

Who did what, and for how long. Hours matter most: they're used for billing. **MUST BE ACCURATE!**

RECURRING DELIVERABLES

Status Reports & Demo Videos

Individual status reports

Due Weeks 2, 7, 11, and 14. This is NOT a group activity; use the template provided on Canvas. Week 14 is slightly different; more of a semester summation.

Team status reports & video

Due Weeks 3, 6, 10, and 13. One page, authored and agreed to (signed) by every team member. Plus, 5-7min Video with accomplishments of each person.



All status reports and videos are due Wednesdays at 11:59pm in their respective weeks so I can review and address any major issues in our Friday class.

KEEP IT HONEST, KEEP IT SHORT

Writing a Good Status Report

Ground rules

- You'll submit some individual and some team reports
- I read (not AI) and respond to issues or specific questions
- Every report needs timesheets. Group status reports needs a short 1–4 min video with every member providing an update. Show me physical work (Screens, HW, etc..) Don’t want to see a talking “head”!
- Should take no more than ~15 minutes to write / create

▲ Watch for these

- Very short report = you did nothing
- Very long report = you also did nothing
- Don't write “Wrote status report” as your accomplishment

Syllabus Walk Through – Roll Call

- Review Each week (1-16)
- Review each deliverable (SRS, DDD, MVP1, Final Project Preso, Final Exam)
- Mid-term (Detailed Design Presentation & Documents)
- Final Project Presentation (Friday December 4th, 2026 (11am – 2:45pm EST)
- Final Exam (50% of question for items discussed during class, 50% from guest speakers)
- **ROLL - CALL**

WHAT YOU'VE BEEN WAITING FOR

Project Selection

Real industry & USF-internal projects, several with an AI component this semester

No IP or NDA allowed on student work. You are an ambassador for USF and the College; these are companies that hire our graduates. USF does not pay for AI/Token access.

- Project descriptions will be walked through in class
- Rank the projects that interest you most
- A random-number process selects order not everyone gets their first choice
- **Real-world lesson #1:** you'll work with people you didn't choose. Find the positive in your teammates.
- **Real-world lesson #2:** you might have to work on a project you really don’t like.

DUE NEXT WEEK

Project Kick-Off Meeting

- Schedule it for early next week, Monday or Tuesday if possible. Sooner is better.
- Send a polite, professional email to your sponsor contact. I'm happy to review it before you send. Email etiquette is a skill many of us are losing.
- Plan for a 2-hour meeting minimum. If your sponsor can't give you 2 hours, ask for a follow-up. They are your CUSTOMER!
- Purpose: collect requirements, define how you'll work all semester, and determine what “success” looks like. How will your customer “Monitor” your progress? Suggested to meet no less than 1 time each week with sponsor for 30min.



First team presentation is due Week 3.

Meet multiple times to review ideas before then.

BEFORE YOU LEAVE TODAY

What's Next

- Get to know your teammates. Exchange contact info, set up a Teams channel
- Schedule your kick-off meeting with your sponsor (or with me, if it's a USF-internal project)
- Study your problem domain. Become a domain expert rapidly!
- Watch the announcements section in Canvas. I post updates there regularly. Assignment details are never changed via announcements.
- Most importantly: start working toward the SRS document and presentation, due Week 5.